ПАРАДИГМЫ И КОНСТРУКЦИИ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

Рубежный контроль №1

Вариант запросов: Г
Номер варианта предметной области: 2
Предметная область: «Школьник — Класс».

Постановка задачи

1. **Г1 — один ко многим.** Вывести классы, названия которых начинаются с буквы «А», и списки обучающихся в них школьников.
2. **Г2 — один ко многим.** Вывести классы с максимальным возрастом школьников в каждом классе, отсортировав результат по этому возрасту. Выбрана сортировка по убыванию.
3. **Г3 — многие ко многим.** Вывести все связанные пары «школьник — класс», отсортированные по названию класса по возрастанию.

Модель данных

Класс Python	Поля
Pupil — школьник	id, surname, age, class_id
SchoolClass — класс	id, name
PupilClass — связь M:M	pupil_id, class_id

Возраст (поле age, в годах) выбран количественным признаком вместо зарплаты в примере задания. Связь 1:M задаётся полем class_id, а связь M:M — отдельным списком объектов PupilClass.

Тестовые данные

Школьник	Возраст	Основной класс (1:M)
Иванов	14	Альфа
Петров	15	Альфа
Сидорова	16	Бета
Алексеева	15	Альтаир
Смирнов	17	Бета

Названия классов условные. В модели M:M сохранены пять основных связей и добавлены две: Иванов — Альтаир и Алексеева — Бета. Классы в этой модели трактуются как учебные группы, которые школьник может посещать одновременно.

Текст программы · 1/2 · main.py

```
"""РК №1. Вариант запросов Г, предметная область №2: Школьник — Класс."""

from operator import itemgetter


class Pupil:
    """Школьник; age — количественный признак, class_id — связь 1:М."""

    def __init__(self, id, surname, age, class_id):
        self.id = id
        self.surname = surname
        self.age = age
        self.class_id = class_id


class SchoolClass:
    """Школьный класс."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class PupilClass:
    """Связь школьника с классом для отношения М:М."""

    def __init__(self, pupil_id, class_id):
        self.pupil_id = pupil_id
        self.class_id = class_id


# Условные названия классов позволяют проверить начало названия на «А».
classes = [
    SchoolClass(1, "Альфа"),
    SchoolClass(2, "Бета"),
    SchoolClass(3, "Альтаир"),
]

pupils = [
    Pupil(1, "Иванов", 14, 1),
    Pupil(2, "Петров", 15, 1),
    Pupil(3, "Сидорова", 16, 2),
    Pupil(4, "Алексеева", 15, 3),
    Pupil(5, "Смирнов", 17, 2),
]

# В модели М:М школьник может посещать несколько учебных классов.
pupils_classes = [
    PupilClass(1, 1),
    PupilClass(2, 1),
    PupilClass(3, 2),
    PupilClass(4, 3),
    PupilClass(5, 2),
    PupilClass(1, 3),
    PupilClass(4, 2),
]
```

Текст программы · 2/2 · main.py

```
def query_g1(pupils, classes):
    """Классы на букву «А» и их школьники; связь 1:М."""
    return {
        c.name: [p.surname for p in pupils if p.class_id == c.id]
        for c in classes
        if c.name.startswith("А")
    }


def query_g2(pupils, classes):
    """Максимальный возраст по классам, по убыванию; связь 1:М."""
    ages_by_class = {
        c.id: [p.age for p in pupils if p.class_id == c.id]
        for c in classes
    }
    # У пустого класса максимальный возраст не определён.
    result = [
        (c.name, max(ages_by_class[c.id]))
        for c in classes
        if ages_by_class[c.id]
    ]
    return sorted(result, key=itemgetter(1), reverse=True)


def query_g3(pupils, classes, links):
    """Все пары (школьник, класс), сортировка по классу; связь М:М."""
    many_to_many = [
        (p.surname, c.name)
        for c in classes
        for link in links
        for p in pupils
        if link.class_id == c.id and link.pupil_id == p.id
    ]
    return sorted(many_to_many, key=itemgetter(1))


def main():
    print("Задание Г1. Классы на букву «А» и их школьники:")
    for name, surnames in query_g1(pupils, classes).items():
        print(f"{name}: {', '.join(surnames) or 'нет школьников'}")

    print("\nЗадание Г2. Максимальный возраст (по убыванию):")
    for name, max_age in query_g2(pupils, classes):
        print(f"{name}: {max_age}")

    print("\nЗадание Г3. Школьники и классы (сортировка по классу):")
    for surname, name in query_g3(pupils, classes, pupils_classes):
        print(f"{surname} — {name}")


if __name__ == "__main__":
    main()
```

Результаты выполнения

Фактический вывод программы при запуске на Python 3.13.0:

```
Задание Г1. Классы на букву «А» и их школьники:
Альфа: Иванов, Петров
Альтаир: Алексеева

Задание Г2. Максимальный возраст (по убыванию):
Бета: 17
Альфа: 15
Альтаир: 15

Задание Г3. Школьники и классы (сортировка по классу):
Алексеева — Альтаир
Иванов — Альтаир
Иванов — Альфа
Петров — Альфа
Сидорова — Бета
Смирнов — Бета
Алексеева — Бета
```

Проверка результатов

Г1: выбраны «Альфа» и «Альтаир»; школьники определены по связи 1:М.

Г2: максимальный возраст в «Бете» — 17, в «Альфе» и «Альтаире» — 15. При равных значениях сохранён исходный порядок классов.

Г3: выведены все семь связей М:М. Порядок классов: «Альтаир», «Альфа», «Бета».

Проверены ожидаемые результаты трёх запросов, соответствие идентификаторов и обработка пустых данных. Класс без школьников не включается в Г2, поскольку максимальный возраст для него не определён.

Запуск

```
python main.py
```

Используются только средства стандартной библиотеки Python: генераторы списков и словарей, max, sorted и operator.itemgetter.